

기계학습 기초 프로젝트 결과보고서

US Accidents 데이터를 활용한 교통사고 **Severity** 예측 모델 및
Streamlit 웹앱 구현



제출일: 2026년 06월 12일

팀 명: 4조

팀 원: 예상형, 윤석민, 강우석, 정시영

Title

US Accidents 데이터를 활용한 교통사고 Severity 예측 모델 및 Streamlit 웹앱 구현

Abstract

본 프로젝트는 Kaggle의 US Accidents (2016–2023) 데이터를 활용하여 교통사고 발생 조건을 기반으로 사고 심각도인 Severity를 예측하는 다중분류 모델을 개발하는 것을 목표로 한다. 사고 발생 시점에서 활용 가능한 시간, 위치, 기상, 도로환경 변수를 중심으로 입력 변수를 구성하였으며, 사고 종료 이후에만 알 수 있는 변수는 정보 누수를 방지하기 위해 제외하였다. 데이터는 Severity 2 클래스가 대부분을 차지하는 불균형 구조를 보였기 때문에 Accuracy뿐 아니라 Balanced Accuracy, Macro F1-score, Weighted F1-score를 함께 사용하였다. 여러 분류 모델과 불균형 처리 전략을 비교한 결과, Validation set의 Macro F1-score 기준으로 Random Forest Balanced 모델이 가장 우수한 성능을 보였으며, 파라미터 조정 후 최종 모델은 RF Balanced min_samples_leaf=5로 선정하였다. 또한 학습된 모델과 전처리 기준을 저장하여 Streamlit 웹앱으로 연결함으로써 사용자가 사고 위치, 기상 정보, 도로환경 정보를 입력하면 Severity 예측 결과와 클래스별 확률을 확인할 수 있도록 구현하였다.

Keywords

Traffic Accident Severity, Multiclass Classification, Random Forest, Class Imbalance, Macro F1-score, Streamlit, US Accidents

1. Introduction

1.1 문제 배경과 필요성

교통사고는 인명 피해뿐만 아니라 교통 혼잡, 도로 운영 효율 저하, 사회적 비용 증가로 이어지는 중요한 교통 문제이다. 사고가 발생하면 해당 도로의 용량이 감소하고, 주변 교통 흐름이 불안정해지며, 사고 처리와 교통 통제를 위한 신속한 의사결정이 필요하다. 따라서

사고 발생 직후 사고의 심각도를 예측할 수 있다면 사고 대응 우선순위 설정, 교통관리 전략 수립, 위험 지역 관리 등에 활용될 수 있다.

사고의 심각도는 단순히 사고 발생 여부만으로 결정되지 않는다. 사고 발생 시간, 위치, 기상 조건, 도로 주변 시설, 주야간 여부 등 다양한 요인이 사고로 인한 교통 지연과 사고 영향 범위에 영향을 줄 수 있다. 예를 들어 출퇴근 시간대, 악천후, 낮은 가시거리, 교차로 또는 신호등 인접 여부는 사고 발생 이후 교통 흐름에 서로 다른 영향을 미칠 수 있다. 따라서 이러한 사고 발생 조건을 종합적으로 활용하여 사고 **Severity**를 예측하는 모델을 구축하는 것은 교통 운영 및 사고 대응 측면에서 의미가 있다.

1.2 해결하고자 하는 과제 정의

Kaggle에서 제공하는 **US Accidents (2016–2023)** 데이터를 활용하여 교통사고 심각도인 **Severity**를 예측하는 지도학습 기반 다중분류 문제를 다룬다. **Severity**는 사고가 교통 흐름에 미친 영향 또는 지연 정도를 나타내는 타깃 변수이며, 1, 2, 3, 4의 네 개 클래스로 구성된다. 본 프로젝트에서는 사고 발생 조건을 입력으로 사용하여 각 사고의 **Severity** 등급을 예측하는 것을 목표로 한다.

입력 변수는 사고 발생 시점에서 확인 가능한 정보 중심으로 구성하였다. 시간 변수로는 사고 발생 월, 요일, 시간대를 사용하였고, 위치 변수로는 위도, 경도, 주, 도시, 카운티, 시간대를 사용하였다. 또한 기상 변수, 주야간 관련 변수, 교차로·신호등·횡단보도 등 도로환경 변수를 함께 활용하였다. 반면 **End_Time**, **Distance(mi)**, **End_Lat**, **End_Lng**, **Description**과 같이 사고 종료 이후에 알 수 있거나 사고 결과와 직접적으로 연결될 수 있는 변수는 정보 누수를 방지하기 위해 제외하였다.

또한 **Severity 2** 클래스가 대부분을 차지했다. 따라서 단순 **Accuracy**만을 기준으로 모델을 평가하면 다수 클래스를 잘 맞추는 모델이 실제보다 우수하게 평가될 수 있다. 결론적으로 **Accuracy**뿐만 아니라 **Balanced Accuracy**, **Weighted F1-score**, **Macro F1-score**를 함께 사용하였으며, 최종 모델 선정의 주요 기준으로 **Macro F1-score**를 사용하였다.

1.3 프로젝트 목표 및 기여 요약

US Accidents 데이터를 활용하여 사고 발생 조건 기반의 **Severity** 다중분류 모델을 구축하는 것이다. 원본 데이터에서 필요한 변수를 선별하고, 시간 파생 변수 생성, 날씨 그룹화, 결측치 처리, 범주형 변수 인코딩, 수치형 변수 표준화 등의 전처리 과정을 수행하였다.

다양한 머신러닝 모델과 클래스 불균형 처리 전략을 비교하여 최종 모델을 선정하는 것이다. 본 프로젝트에서는 **Dummy Classifier**, **Logistic Regression**, **Decision Tree**, **Random Forest**, **HistGradientBoosting**, **XGBoost**, **LightGBM**, **Dense Neural Network** 등을 비교하였다. 또한 클래스 불균형 문제를 완화하기 위해 **class weight**, **undersampling**, **oversampling**, **SMOTE** 등의 방법을 실험하였다. 최종적으로 **Validation set**의 **Macro F1-score**를 기준으로 가장 우수한 모델을 선정하였다.

또한 학습된 모델을 실제 사용 가능한 형태로 구현하는 것이다. 이를 위해 최종 모델과 전처리 정보를 저장하고, **Streamlit** 기반 웹앱을 구현하였다. 웹앱에서는 사용자가 미국 지도에서 사고 위치를 선택하면 위도와 경도가 자동 입력되고, 가장 가까운 기존 사고 데이터를 기준으로 주, 도시, 카운티, 시간대가 자동 설정된다. 이후 기상 정보와 도로환경 정보를 입력하면 예측 **Severity**와 클래스별 예측 확률을 확인할 수 있다.

따라서 단순히 사고 데이터를 분석하는 데 그치지 않고, 정보 누수를 고려한 현실적인 예측 문제 정의, 클래스 불균형을 고려한 모델 평가, 그리고 최종 모델의 웹앱 구현까지 하나의 파이프라인으로 연결했다는 점에 있다.

2. Related Work

2.1 기존 연구 및 유사 접근 방식

교통사고 심각도 예측은 사고 발생 조건과 사고 결과 사이의 관계를 학습하여 사고의 위험 수준을 분류하는 문제로 다루어져 왔다. 기존 연구에서는 사고 발생 시간, 위치, 기상 조건, 도로 기하구조, 교차로 여부, 노면 상태, 운전자 및 차량 관련 특성 등을 주요 입력 변수로 사용하였다. 이러한 변수들은 사고가 발생했을 때 사고의 피해 규모나 교통 흐름에 미치는

영향을 설명하는 데 활용될 수 있으며, 최근에는 머신러닝과 AI 기반 방법을 활용해 사고 심각도를 예측하려는 연구도 진행되고 있다 [2].

기존 기계학습 기반 사고 심각도 예측에서는 Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, XGBoost, LightGBM과 같은 모델이 주로 사용된다 [3]. 이 중 Random Forest와 XGBoost 계열 모델은 비선형 관계를 학습할 수 있고, 변수 중요도를 통해 주요 영향 요인을 해석할 수 있다는 장점이 있다. 특히 사고 데이터는 시간, 위치, 기상, 도로환경 등 다양한 유형의 변수가 함께 포함된 정형 데이터이므로, 트리 기반 앙상블 모델이 비교적 안정적으로 활용될 수 있다.

또한 사고 심각도 예측에서는 클래스 불균형 문제가 중요한 이슈로 다루어진다. 실제 사고 데이터에서는 경미하거나 중간 수준의 사고가 대부분을 차지하고, 매우 심각한 사고는 상대적으로 적게 나타나는 경우가 많다. 이러한 경우 Accuracy만을 기준으로 모델을 평가하면 다수 클래스를 잘 예측하는 모델이 과대평가될 수 있다. 따라서 기존 연구 및 유사 프로젝트에서는 Balanced Accuracy, Recall, F1-score, Macro F1-score와 같은 지표를 함께 사용하거나, class weight, undersampling, oversampling, SMOTE 등의 불균형 처리 기법을 적용한다 [4].

US Accidents 데이터셋을 활용한 유사 프로젝트들도 사고 발생 시간, 위치, 기상 조건, 도로 주변 시설 정보를 활용하여 사고 Severity를 예측하는 방식으로 진행되는 경우가 많다. 이 데이터셋은 미국 전역의 대규모 사고 데이터를 포함하고 있어 다양한 지역과 사고 조건을 반영할 수 있다는 장점이 있다. 하지만 데이터 규모가 크고 클래스 불균형이 존재하기 때문에, 모델 학습 전 샘플링, 변수 선택, 결측치 처리, 범주형 변수 인코딩 등의 전처리 과정이 중요하다.

2.2 기존 방법과 본 프로젝트의 차이점

기존의 사고 심각도 예측 접근은 주로 모델 성능 향상에 초점을 두고 다양한 알고리즘을 비교하거나, 특정 모델의 예측 정확도를 높이는 방식으로 진행된다. 반면 본 프로젝트는 실제 사고 발생 시점에서 사용할 수 있는 정보만을 입력 변수로 구성했다는 점에서 차이가 있다. End_Time, Distance(mi), End_Lat, End_Lng, Description과 같이 사고 종료 이후에 알

수 있거나 사고 결과와 직접적으로 연결될 수 있는 변수는 정보 누수 가능성이 있다고 판단하여 제외하였다. 이를 통해 실제 예측 상황에 더 가까운 문제 설정을 구성하였다.

또한 단순 **Accuracy** 중심의 평가가 아니라 클래스 불균형을 고려한 **Macro F1-score**를 최종 모델 선정의 주요 기준으로 사용하였다. **US Accidents** 데이터는 **Severity 2** 클래스가 큰 비중을 차지하므로, **Accuracy**만 사용하면 다수 클래스를 잘 맞추는 모델이 좋은 모델로 평가될 수 있다. 본 프로젝트에서는 **Accuracy**, **Balanced Accuracy**, **Weighted F1-score**를 함께 확인하되, 각 클래스의 성능을 동일한 비중으로 반영하는 **Macro F1-score**를 중심으로 모델을 비교하였다.

모델링 측면에서는 **Dummy Classifier**, **Logistic Regression**, **Decision Tree**, **Random Forest**, **HistGradientBoosting**, **XGBoost**, **LightGBM**, **Dense Neural Network** 등 다양한 모델을 비교하였다. 또한 **class weight**, **undersampling**, **oversampling**, **SMOTE** 등의 불균형 처리 방법을 추가로 실험하여 단순 모델 비교에 그치지 않고 데이터 불균형 문제를 완화하기 위한 접근을 함께 검토하였다.

마지막으로 모델 학습과 평가에서 끝나지 않고, 최종 모델을 **Streamlit** 웹앱으로 구현했다는 점에서 차별성을 가진다. 사용자는 웹앱에서 미국 지도 기반으로 사고 위치를 선택하고, 기상 정보와 도로환경 정보를 입력한 뒤 예측 **Severity**와 클래스별 확률을 확인할 수 있다. 즉, 본 프로젝트는 데이터 분석, 모델 학습, 성능 평가, 웹 기반 서비스 구현까지 하나의 파이프라인으로 연결한 실습형 프로젝트라는 점에서 의의가 있다.

3. Approach

3.1 전체 문제 해결 전략

교통사고 발생 조건을 입력으로 받아 사고 심각도인 **Severity**를 예측하는 다중분류 문제로 정의하였다. 전체 문제 해결 과정은 데이터 수집 및 샘플링, 변수 선택, 전처리, 탐색적 데이터 분석, 모델 학습, 성능 평가, 최종 모델 선정, 웹앱 구현 순서로 진행하였다.

먼저 **Kaggle**의 **US Accidents** 데이터를 불러온 뒤, 전체 원본 데이터의 크기가 매우 크기 때문에 계산 효율성을 고려하여 **Severity** 기준 층화 샘플링을 수행하였다. 이후 사고 발생

시점에서 확인 가능한 변수만을 중심으로 입력 변수를 구성하였다. 사고 종료 이후에 알 수 있는 변수나 사고 결과와 직접적으로 연결될 수 있는 변수는 정보 누수 가능성을 고려하여 제외하였다.

모델링 단계에서는 여러 머신러닝 모델을 학습시키고, **Validation set**의 성능을 비교하였다. 본 데이터는 **Severity 2**가 대부분을 차지하는 불균형 데이터이므로 단순 **Accuracy**가 아니라 **Macro F1-score**를 주요 기준으로 사용하였다. 최종적으로 **Validation set**에서 가장 높은 **Macro F1-score**를 보인 모델을 최종 모델로 선정하고, **Test set**에서 최종 성능을 확인하였다.

마지막으로 최종 모델과 전처리 정보를 저장한 뒤, **Streamlit** 기반 웹앱에 연결하였다. 이를 통해 사용자가 사고 위치, 기상 정보, 도로환경 정보를 입력하면 학습 당시와 동일한 전처리 과정을 거쳐 예측 **Severity**와 클래스별 확률을 출력할 수 있도록 구성하였다.

3.2 데이터 전처리 방법

데이터 전처리는 모델 학습에 적합한 입력 형태를 만들고, 실제 예측 상황에서 사용할 수 있는 변수만을 반영하기 위해 수행하였다. 먼저 원본 데이터에서 사고 발생 시간, 위치, 기상, 도로환경, 주야간 관련 변수와 타깃 변수인 **Severity**를 선택하였다. 반면 **End_Time**, **Distance(mi)**, **End_Lat**, **End_Lng**, **Description**은 사고가 종료된 이후에 알 수 있거나 사고 결과와 직접적으로 연결될 수 있으므로 제외하였다.

시간 변수의 경우 **Start_Time**을 날짜형으로 변환한 뒤 **Month**, **DayOfWeek**, **Hour**를 파생 변수로 생성하였다. 이를 통해 사고가 발생한 월, 요일, 시간대가 **Severity** 예측에 반영될 수 있도록 하였다. 기상 변수에서는 다양한 문자열 형태로 존재하는 **Weather_Condition**을 단순화하여 **Weather_Group**으로 변환하였다. 예를 들어 맑음 계열은 **Clear**, 흐림 계열은 **Cloudy**, 비 계열은 **Rain**, 눈 계열은 **Snow**, 안개 계열은 **Fog** 등으로 그룹화하였다.

결측치 처리는 변수 유형에 따라 다르게 적용하였다. 수치형 변수는 이상치의 영향을 줄이기 위해 학습 데이터 기준 중앙값으로 대체하였고, 범주형 변수는 학습 데이터 기준 최빈값으로 대체하였다. **City**와 **County**처럼 고유값이 많은 범주형 변수는 학습 데이터 기준 상위 30개 값만 유지하고 나머지는 **Other**로 처리하였다. 이는 범주 수가 지나치게 증가하여 모델 입력 차원이 커지는 문제를 줄이기 위한 처리이다.

도로환경 관련 **Boolean** 변수는 0과 1의 정수형 변수로 변환하였다. 범주형 변수는 **One-Hot Encoding**을 적용하여 모델이 학습할 수 있는 수치형 형태로 변환하였고, 연속형 수치 변수는 **StandardScaler**를 사용하여 표준화하였다. 웹앱에서도 학습 시 저장한 중앙값, 최빈값, 상위 범주, 인코딩 컬럼, 스케일러 정보를 그대로 사용하여 학습 데이터와 동일한 기준으로 입력값이 전처리되도록 하였다.

처리 항목	적용 방법	목적
데이터 샘플링	Severity 기준 층화 샘플링	클래스 비율 유지 및 계산 효율 확보
시간 변수	Start_Time 에서 Month, DayOfWeek, Hour 생성	사고 발생 시점의 시간적 특성 반영
날씨 변수	Weather_Condition 을 Weather_Group 으로 단순화	희소한 날씨 범주 완화
결측치 처리	수치형 중앙값, 범주형 최빈값	결측값으로 인한 학습 오류 방지
고유값 많은 범주	City, County 상위 30개 유지, 나머지 Other	입력 차원 증가 완화
범주형 변수	One-Hot Encoding	모델 학습 가능한 수치형 변환
수치형 변수	StandardScaler 표준화	변수 단위 차이 완화
웹앱 입력	저장된 artifact 기준 동일 전처리	학습-예측 환경 일관성 유지

표 1. 주요 전처리 방법 요약

3.3 사용 모델 및 알고리즘 설명

다양한 분류 모델을 비교하여 최종 모델을 선정하였다. 먼저 **Dummy Classifier**를 사용하여 기준 성능을 확인하였다. **Dummy Classifier**는 데이터의 다수 클래스를 중심으로 단순 예측을 수행하므로, 실제 모델이 최소한 어느 정도 기준 성능을 넘어서는지 확인하기 위한 베이스라인 역할을 한다.

Logistic Regression은 선형 기반의 분류 모델로, 입력 변수와 클래스 간의 선형적 관계를 학습한다. 구조가 단순하고 해석이 비교적 쉬운 장점이 있지만, 사고 **Severity**처럼 시간, 위치, 기상, 도로환경 간의 복합적인 비선형 관계가 존재할 수 있는 문제에서는 성능에 한계가 있을 수 있다.

Decision Tree는 조건 분기를 통해 데이터를 분류하는 모델이다. 변수 간 비선형 관계를 표현할 수 있고 해석이 쉽다는 장점이 있지만, 단일 트리 구조이므로 과적합에 취약할 수 있다. **Random Forest**는 여러 개의 **Decision Tree**를 학습한 뒤 결과를 종합하는 앙상블

모델로, 단일 **Decision Tree**보다 일반화 성능이 안정적이다. 본 프로젝트에서는 클래스 불균형을 고려하기 위해 `class_weight=balanced`를 적용한 **Random Forest** 모델도 함께 실험하였다.

HistGradientBoosting, **XGBoost**, **LightGBM**은 **Gradient Boosting** 계열 모델로, 이전 모델의 오차를 보완하는 방식으로 순차적으로 모델을 학습한다. 이들은 정형 데이터에서 높은 예측 성능을 보이는 경우가 많으며, 복잡한 비선형 관계를 학습할 수 있다. 또한 클래스 가중치 또는 샘플 가중치를 적용하여 불균형 데이터에 대응할 수 있다.

Dense Neural Network는 완전연결층 기반의 간단한 신경망 모델로 구성하였다. 은닉층과 **Dropout**을 사용하여 비선형 관계를 학습하도록 설계하였으나, 본 프로젝트의 데이터는 이미지나 텍스트가 아닌 정형 데이터이므로 트리 기반 모델에 비해 성능이 제한적으로 나타날 수 있다.

클래스 불균형 문제를 완화하기 위해 추가 실험도 수행하였다. `class_weight=balanced`는 클래스 빈도에 반비례하여 가중치를 부여하는 방식이며, 소수 클래스의 오분류에 더 큰 패널티를 주도록 한다. **Undersampling**은 다수 클래스의 샘플 수를 줄여 클래스 비율을 완화하는 방법이고, **oversampling**은 소수 클래스 샘플을 복제하여 학습 데이터의 균형을 맞추는 방법이다. **SMOTE**는 소수 클래스의 기존 샘플을 바탕으로 새로운 가상 샘플을 생성하는 방식이다. 이러한 방법들을 비교하여 **Macro F1-score** 기준으로 최종 모델을 선정하였다.

3.4 시스템 및 파이프라인 구성

전체 시스템은 모델 개발 파이프라인과 웹앱 예측 파이프라인으로 나눌 수 있다. 모델 개발 파이프라인에서는 원본 데이터를 불러오고, 층화 샘플링을 통해 분석용 데이터를 생성한 뒤, 전처리와 모델 학습을 수행한다. 이후 **Validation set**을 기준으로 여러 모델의 성능을 비교하고, 최종 모델을 선정한 뒤 **Test set**으로 최종 성능을 평가한다. 최종 모델과 전처리 정보를 **joblib** 파일로 저장하여 웹앱에서 동일하게 사용할 수 있도록 하였다.

웹앱 예측 파이프라인에서는 사용자가 **Streamlit** 화면에서 사고 정보를 입력한다. 사용자는 지도에서 사고 위치를 선택할 수 있으며, 선택된 위도와 경도를 기준으로 가장 가까운 기존 사고 데이터를 찾아 **State, City, County, Timezone**을 자동 설정한다. 이후 사용자가 기상 정보, 주야간 정보, 도로환경 정보를 입력하고 예측 버튼을 누르면, 웹앱은 저장된 전처리

artifact를 불러와 학습 당시와 동일한 방식으로 입력값을 변환한다. 변환된 입력값은 최종 모델에 전달되고, 모델은 예측 **Severity**와 클래스별 예측 확률을 출력한다.

이러한 구성은 데이터 분석과 모델 학습 결과를 실제 사용 가능한 형태로 연결한다는 점에서 의미가 있다. 단순히 노트북에서 모델 성능을 확인하는 데 그치지 않고, 사용자가 직접 입력한 사고 조건에 대해 예측 결과를 확인할 수 있도록 구현함으로써 프로젝트의 활용 가능성을 높였다.

전체 문제 해결 및 시스템 파이프라인

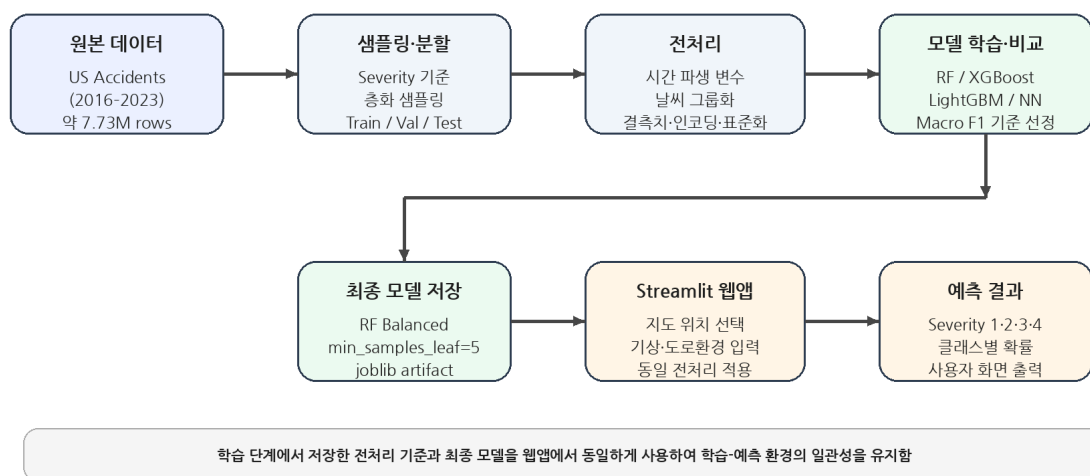


그림 1. 전체 문제 해결 및 시스템 파이프라인

4. Experiment

4.1 데이터셋 설명

Kaggle에서 제공하는 **US Accidents (2016–2023)** 데이터셋을 사용하였다[1]. 해당 데이터셋은 미국 전역에서 수집된 교통사고 데이터를 포함하며, 사고 발생 시간, 위치, 기상 조건, 도로 주변 시설, 주야간 정보, 사고 심각도 등의 변수를 포함한다. 본 프로젝트의 예측 대상 변수는 **Severity**이며, 이는 사고로 인한 교통 지연 또는 교통 흐름 영향 정도를 나타내는 등급형 변수이다. **Severity**는 1, 2, 3, 4의 네 개 클래스로 구성되며, 숫자가 클수록 사고의 영향이 큰 것으로 해석된다.

원본 데이터는 약 773만 개의 사고 데이터를 포함하고 있어 전체 데이터를 모두 사용하기에는 학습 시간과 계산 자원 부담이 크다. 따라서 본 프로젝트에서는 Severity 클래스 비율을 유지하기 위해 층화 샘플링을 적용하여 100,000개의 데이터를 추출하였다. 최종적으로 사고 발생 시점에서 확인 가능한 시간, 위치, 기상, 도로환경, 주야간 관련 변수를 중심으로 34개의 변수를 선택하였다.

데이터셋은 Severity 2 클래스가 대부분을 차지하는 불균형 구조를 보였다. 이러한 특성 때문에 단순 Accuracy만으로 모델을 평가할 경우 다수 클래스 중심의 예측 성능이 과대평가될 수 있다. 따라서 본 프로젝트에서는 클래스별 성능을 균형 있게 반영할 수 있는 평가 지표를 함께 사용하였다.

항목	내용
Dataset	US Accidents (2016–2023)
Source	Kaggle
Target Variable	Severity
Task Type	Multiclass Classification
Original Data Size	약 7.73M rows
Sample Size	100,000 rows
Number of Classes	4 classes
Selected Features	34 variables
Split Ratio	Train 70% / Validation 15% / Test 15%
Main Metric	Macro F1-score

표 2. 데이터셋 및 실험 설정 요약

4.2 실험 환경 및 설정

실험은 Python 기반의 Jupyter Notebook 환경에서 수행하였다. 데이터 처리에는 pandas와 numpy를 사용하였고, 모델 학습 및 평가에는 scikit-learn을 중심으로 xgboost, lightgbm, tensorflow를 추가로 사용하였다. 학습된 최종 모델과 전처리 정보는 joblib 형식으로 저장하였으며, 이후 Streamlit 웹앱에서 동일한 전처리 기준과 모델을 불러와 예측에 활용하였다.

모델 비교를 위해 Dummy Classifier, Logistic Regression, Decision Tree, Random Forest, HistGradientBoosting, XGBoost, LightGBM, Dense Neural Network를 사용하였다. 또한 클래스 불균형을 고려하기 위해 class_weight=balanced, manual class weight,

undersampling, oversampling, SMOTE 등의 방법을 실험하였다. 모델 선정은 Validation set의 Macro F1-score를 주요 기준으로 수행하였다.

실험의 재현성을 높이기 위해 데이터 샘플링과 데이터 분할 과정에서 random state를 고정하였다. 또한 전처리 과정에서 결측치 대체 기준, 상위 범주 목록, One-Hot Encoding 이후의 컬럼 목록, StandardScaler 객체를 저장하여 학습 단계와 웹앱 예측 단계에서 동일한 전처리 방식이 적용되도록 하였다.

4.3 학습/검증/테스트 분할 방식

전체 샘플링 데이터 100,000개를 Train set, Validation set, Test set으로 분할하였다. 분할 비율은 Train 70%, Validation 15%, Test 15%로 설정하였다. Train set은 모델 학습에 사용하였고, Validation set은 모델 비교와 하이퍼파라미터 선택에 사용하였다. Test set은 최종 선택된 모델의 일반화 성능을 확인하기 위한 최종 평가에 사용하였다.

데이터 분할 과정에서는 Severity 클래스 비율이 각 데이터셋에서 최대한 유지되도록 stratify 옵션을 적용하였다. 이는 원본 데이터가 불균형 구조를 가지고 있기 때문에, 특정 분할에서 소수 클래스가 지나치게 적게 포함되는 문제를 줄이기 위한 처리이다.

다만 본 프로젝트의 분할 방식은 무작위 층화 분할이므로, 유사한 지역이나 시간대의 사고가 Train, Validation, Test set에 함께 포함될 수 있다. 따라서 본 실험의 성능은 동일한 데이터 분포 내에서의 일반화 성능으로 해석하는 것이 적절하다. 실제 지역 확장 성능이나 미래 시점에 대한 예측 성능을 더 엄밀히 검증하기 위해서는 향후 State 단위 분리 검증이나 시간 기준 분리 검증을 추가로 수행할 필요가 있다.

4.4 평가 지표 정의

모델 성능을 평가하기 위해 Accuracy, Balanced Accuracy, Macro F1-score, Weighted F1-score를 사용하였다. Accuracy는 전체 샘플 중 정답을 맞힌 비율을 나타내며, 모델의 전반적인 예측 정확도를 확인하는 데 사용된다. 그러나 본 데이터처럼 특정 클래스가 대부분을 차지하는 불균형 데이터에서는 Accuracy만으로 모델 성능을 판단하기 어렵다.

Balanced Accuracy는 각 클래스별 Recall을 평균한 값으로, 클래스 불균형 상황에서도 각 클래스의 예측 성능을 비교적 균형 있게 반영할 수 있다. Macro F1-score는 각 클래스의 F1-score를 동일한 비중으로 평균한 값이다. 따라서 다수 클래스뿐만 아니라 소수 클래스의

Precision과 Recall까지 함께 고려할 수 있다. **Weighted F1-score**는 각 클래스의 데이터 수를 가중치로 반영하여 **F1-score**를 평균한 값으로, 전체 데이터 분포를 고려한 성능을 확인하는데 사용된다.

최종 모델 선정의 주요 지표는 **Macro F1-score**로 설정하였다. 이는 **Severity 2**가 대부분을 차지하는 데이터 구조에서 다수 클래스에 편향된 모델을 피하고, **Severity 1**, **Severity 3**, **Severity 4**와 같은 상대적으로 적은 클래스의 예측 성능도 함께 고려하기 위함이다.

평가 지표	정의	사용 목적
Accuracy	전체 샘플 중 정답을 맞힌 비율	전체적인 예측 정확도 확인
Balanced Accuracy	클래스별 Recall의 평균	클래스 불균형을 고려한 성능 확인
Macro F1-score	클래스별 F1-score의 단순 평균	소수 클래스 성능까지 균등 반영
Weighted F1-score	클래스별 F1-score를 support 기준 가중 평균	실제 데이터 분포를 반영한 전체 성능 확인

표 3. 평가 지표 정의

5. Results

5.1 모델 비교 결과

여러 분류 모델과 클래스 불균형 처리 방법을 비교하여 최종 모델을 선정하였다. 모델 비교의 주요 기준은 **Validation set**의 **Macro F1-score**로 설정하였다. 이는 **Severity 2** 클래스가 대부분을 차지하는 데이터 구조에서 다수 클래스 중심의 **Accuracy**만으로 모델을 선택하는 문제를 줄이고, 각 **Severity** 클래스의 예측 성능을 균등하게 반영하기 위함이다.

초기 모델로는 **Dummy Classifier**, **Logistic Regression**, **Decision Tree**, **Random Forest**, **HistGradientBoosting**을 사용하였다. 이후 클래스 불균형 문제를 고려하기 위해 **class_weight=balanced**를 적용한 **Random Forest** 계열 모델을 추가로 실험하였다. 또한 **XGBoost Weighted**, **LightGBM Balanced**, **Dense Neural Network** 모델도 비교하였다.

Validation set 기준으로 가장 높은 **Macro F1-score**를 보인 모델은 **RF Balanced min_samples_leaf=5**였다. 해당 모델의 **Validation Macro F1-score**는 **0.4095**로 나타났으며, 최종 모델로 선정하였다. **Random Forest** 계열 모델은 시간, 위치, 기상, 도로환경 변수 사이의 비선형 관계를 안정적으로 학습할 수 있었고, **class weight** 설정을 통해 클래스 불균형 문제를 일부 완화할 수 있었다.

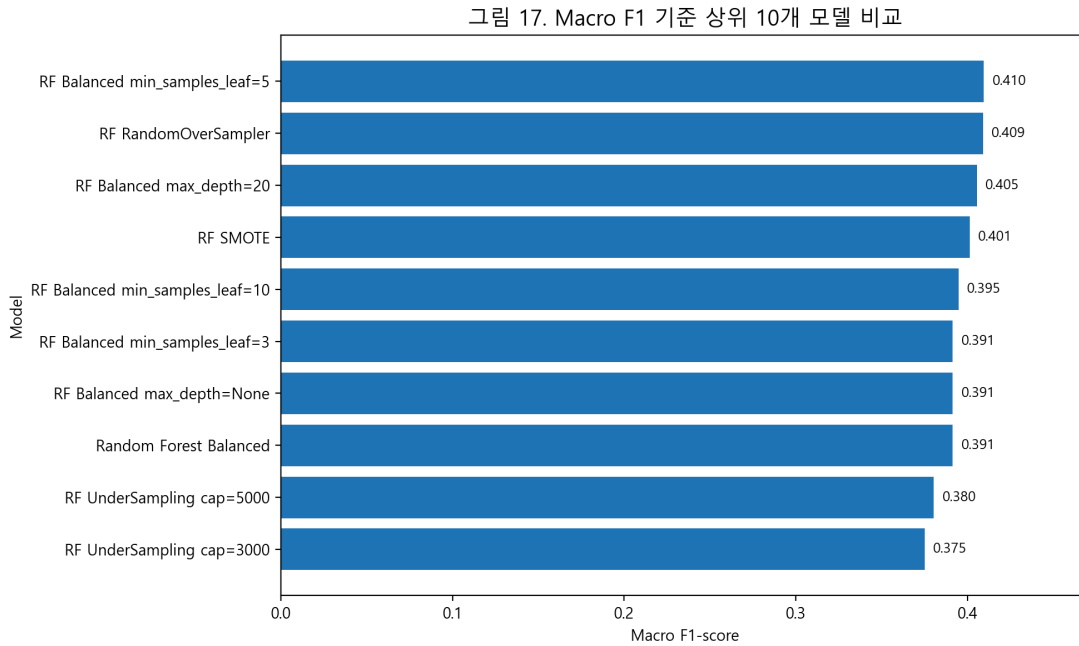


그림 2. Validation Macro F1-score 기준 상위 10개 모델 비교

5.2 클래스 불균형 처리 실험 결과

Severity 2 클래스가 매우 큰 비중을 차지하기 때문에, 클래스 불균형 처리 방법에 따른 성능 차이를 추가로 비교하였다. 실험에는 undersampling, manual class weight, RandomOverSampler, SMOTE 등을 적용하였다. 이 중 RandomOverSampler를 적용한 Random Forest 모델이 불균형 처리 실험 내에서는 가장 높은 Macro F1-score를 보였으나, 전체 모델 후보를 통합하여 비교했을 때는 RF Balanced min_samples_leaf=5가 가장 우수한 성능을 보였다.

불균형 처리 실험 결과는 단순히 소수 클래스 샘플 수를 늘리거나 다수 클래스 샘플 수를 줄이는 방식이 항상 최종 성능 향상으로 이어지지는 않음을 보여준다. 특히 원-핫 인코딩된 범주형 변수가 많은 데이터에서는 SMOTE와 같은 가상 샘플 생성 방법을 적용할 때 해석상 주의가 필요하다. 따라서 본 프로젝트에서는 Validation Macro F1-score를 기준으로 가장 안정적인 성능을 보인 class weight 기반 Random Forest 모델을 최종 모델로 선정하였다.

Model	Macro F1-score
RF RandomOverSampler	0.4090
RF SMOTE	0.4013
RF UnderSampling cap=5000	0.3804

RF UnderSampling cap=3000	0.3750
RF ManualWeight 3	0.3586
RF ManualWeight 2	0.3442
RF UnderSampling cap=10000	0.3377
RF ManualWeight 1	0.3369

표 4. 클래스 불균형 처리 실험 결과

5.3 최종 모델의 Test set 성능

최종 모델인 RF Balanced min_samples_leaf=5를 Test set에 적용한 결과, Accuracy는 0.7415, Balanced Accuracy는 0.4417, Macro F1-score는 0.4185, Weighted F1-score는 0.7546으로 나타났다. Accuracy와 Weighted F1-score는 비교적 높게 나타났지만, Balanced Accuracy와 Macro F1-score는 상대적으로 낮았다. 이는 모델이 다수 클래스인 Severity 2에 대해서는 비교적 안정적인 예측 성능을 보였지만, 데이터 수가 적은 Severity 1과 Severity 4에서는 예측 성능이 낮았기 때문이다.

클래스별 성능을 살펴보면 Severity 2의 F1-score는 0.8403으로 가장 높게 나타났다. 반면 Severity 1의 F1-score는 0.2006, Severity 4의 F1-score는 0.1630으로 낮게 나타났다. Severity 3의 F1-score는 0.4701로 중간 수준의 성능을 보였다. 이러한 결과는 클래스별 데이터 수 차이가 모델 성능에 큰 영향을 미쳤음을 보여준다.

Metric	Value
Accuracy	0.7415
Balanced Accuracy	0.4417
Macro F1-score	0.4185
Weighted F1-score	0.7546

표 5. 최종 모델 Test set 성능

Severity	Precision	Recall	F1-score	Support
1	0.1742	0.2366	0.2006	131
2	0.8787	0.8052	0.8403	11950
3	0.4070	0.5563	0.4701	2522
4	0.1576	0.1688	0.1630	397

Macro avg	0.4044	0.4417	0.4185	15000
Weighted avg	0.7742	0.7415	0.7546	15000

표 6. 클래스별 **Classification Report**

그림 18. Confusion Matrix - RF Balanced min_samples_leaf=5

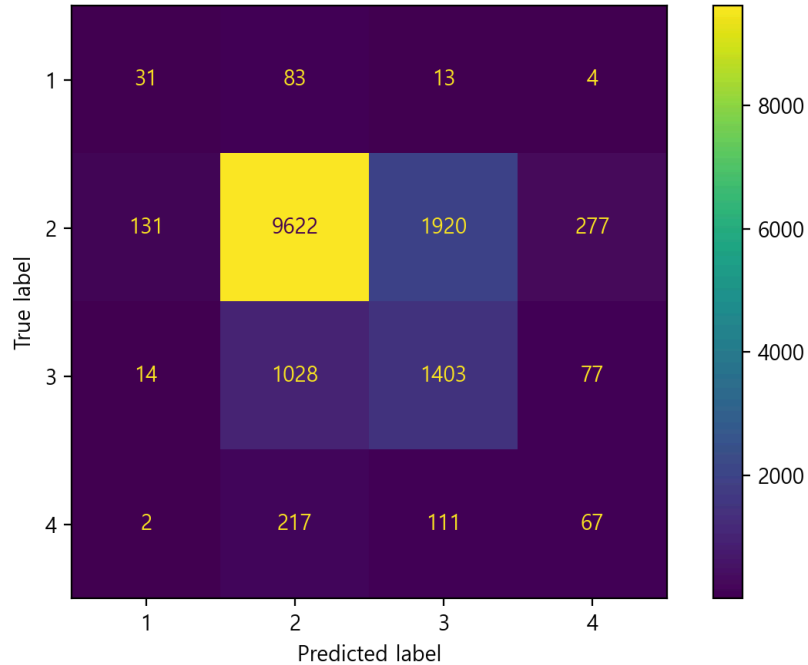


그림 3. 최종 모델의 **Test set Confusion Matrix**

5.4 변수 중요도 결과

최종 모델의 변수 중요도를 확인한 결과, Start_Lng, Start_Lat, Wind_Chill(F), Month, Pressure(in), Temperature(F), Hour, Humidity(%) 등이 상위 변수로 나타났다. 특히 위도와 경도 변수가 가장 높은 중요도를 보였는데, 이는 사고 **Severity**가 지역적 특성과 관련될 가능성이 있음을 의미한다. 또한 기상 변수와 시간 변수가 상위권에 포함되어 사고 발생 환경이 **Severity** 예측에 중요한 역할을 한 것으로 해석할 수 있다.

다만 변수 중요도는 **Random Forest** 모델 내부에서 계산된 상대적 중요도이므로, 해당 변수가 사고 심각도의 직접적인 원인이라고 단정할 수는 없다. 예를 들어 위치 변수의 중요도가 높게 나타난 것은 특정 지역의 사고 기록 방식, 도로망 특성, 교통량 차이 등이 함께 반영된 결과일 수 있다. 따라서 변수 중요도 결과는 모델 해석을 위한 참고 지표로 활용하는 것이 적절하다.

Rank	Feature	Importance
------	---------	------------

1	Start_Lng	0.0759
2	Start_Lat	0.0742
3	Wind_Chill(F)	0.0710
4	Month	0.0624
5	Pressure(in)	0.0603
6	Temperature(F)	0.0572
7	Hour	0.0561
8	Humidity(%)	0.0486
9	Precipitation_NA	0.0403
10	Wind_Speed(mph)	0.0380

표 7. 최종 모델 주요 변수 중요도 Top 10

5.5 Streamlit 웹앱 구현 결과

최종 모델을 실제 사용 가능한 형태로 확장하기 위해 **Streamlit** 기반 웹앱을 구현하였다. 웹앱은 사용자가 사고 발생 조건을 직접 입력하면 학습된 모델이 해당 사고의 **Severity**를 예측하고, 클래스별 예측 확률을 함께 출력하도록 구성하였다.

웹앱의 주요 기능은 지도 기반 위치 선택, 사고 기본 정보 입력, 기상 정보 입력, 도로환경 정보 입력, 최종 **Severity** 예측으로 구성된다. 사용자는 **Folium** 기반 미국 지도에서 사고 위치를 클릭할 수 있으며, 클릭한 좌표를 기준으로 위도와 경도가 자동 입력된다. 또한 선택된 좌표와 가장 가까운 기존 사고 데이터를 참조하여 **State, City, County, Timezone** 정보가 자동 설정되도록 하였다. 이를 통해 사용자가 위치 정보를 모두 직접 입력해야 하는 부담을 줄였다.

예측 과정에서는 노트북 학습 단계에서 저장한 전처리 정보와 최종 모델을 **joblib artifact**로 불러온다. 사용자가 입력한 사고 정보는 학습 당시와 동일한 기준으로 시간 파생 변수 생성, 날씨 그룹화, 결측치 처리, 범주형 변수 인코딩, 수치형 변수 표준화 과정을 거친다. 이후 전처리된 입력값이 최종 모델인 **RF Balanced min_samples_leaf=5**에 전달되고, 웹 화면에는 예측된 **Severity** 등급과 클래스별 예측 확률이 출력된다.

Streamlit 웹앱 구현 결과, 본 프로젝트는 노트북에서 모델 성능을 확인하는 단계에 그치지 않고, 사용자가 직접 사고 조건을 입력하여 예측 결과를 확인할 수 있는 형태로 확장되었다. 이는 모델 개발 결과를 실제 활용 가능한 인터페이스와 연결했다는 점에서 의의가 있다. 다만 현재 웹앱은 기상 정보와 도로환경 정보를 사용자가 직접 입력해야 하며, 행정구역 정보도 가장 가까운 기존 사고 데이터를 기준으로 설정되므로 실제 행정구역과 완전히

일치하지 않을 수 있다. 향후에는 날씨 API, 지도 API, 교통량 API와 연계하여 입력 자동화와 예측 편의성을 개선할 필요가 있다.

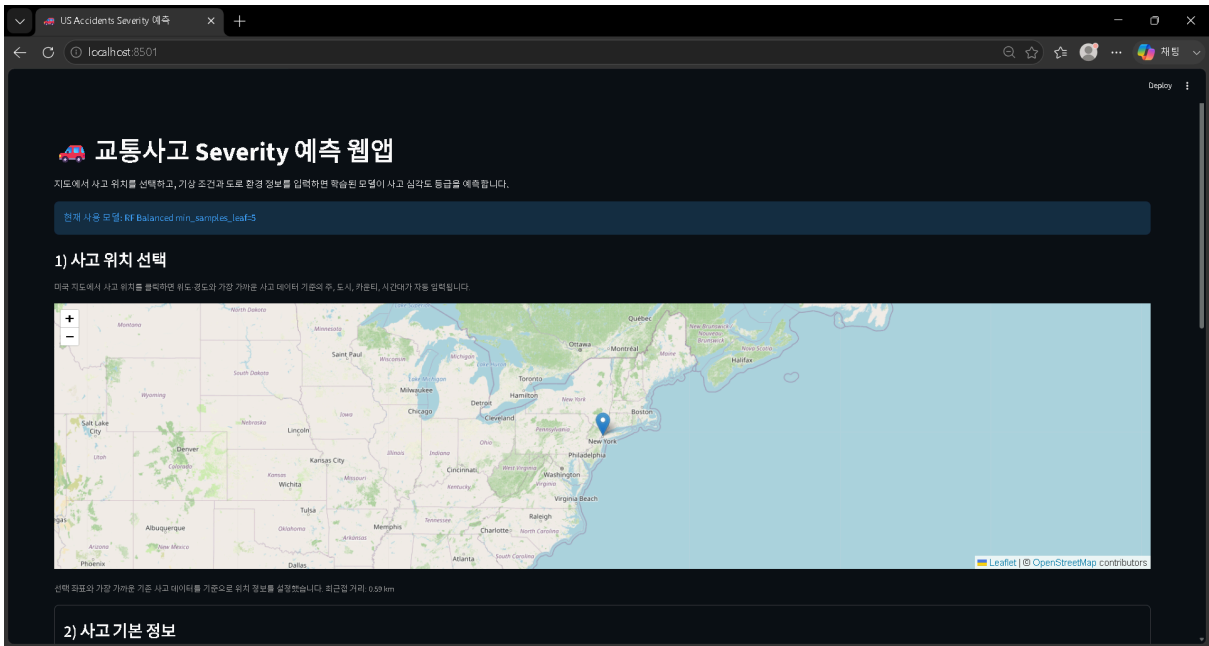


그림 4. Streamlit 웹앱의 지도 기반 사고 위치 선택 화면

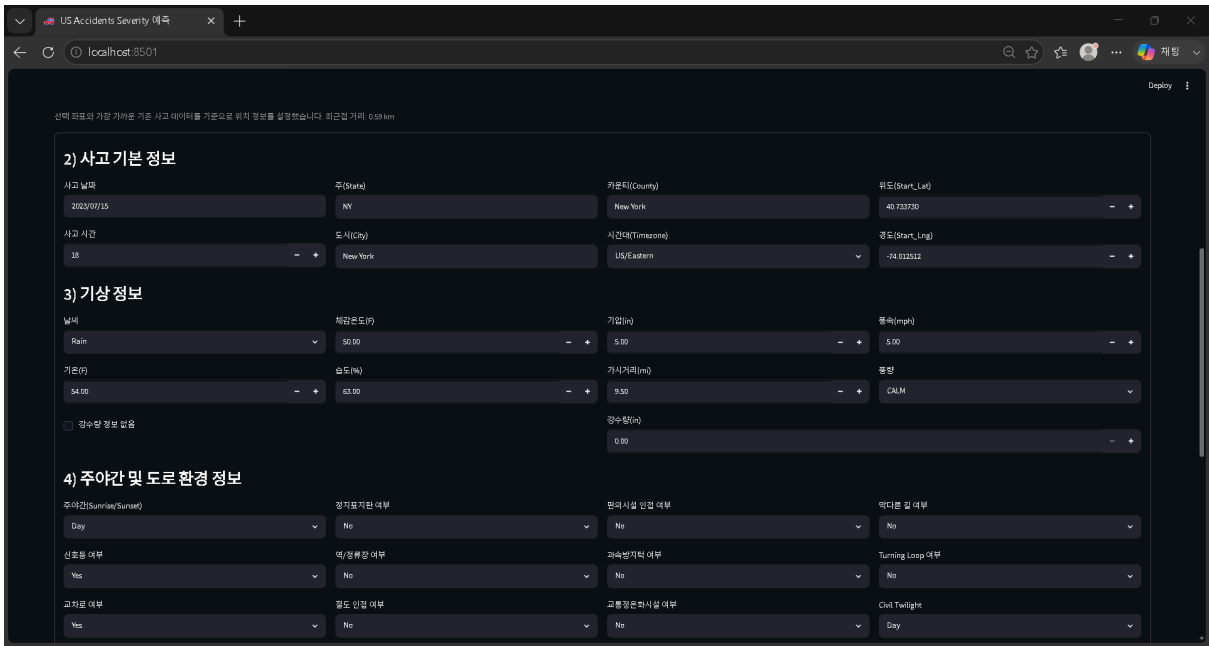


그림 5. Streamlit 웹앱의 사고 조건 입력 화면

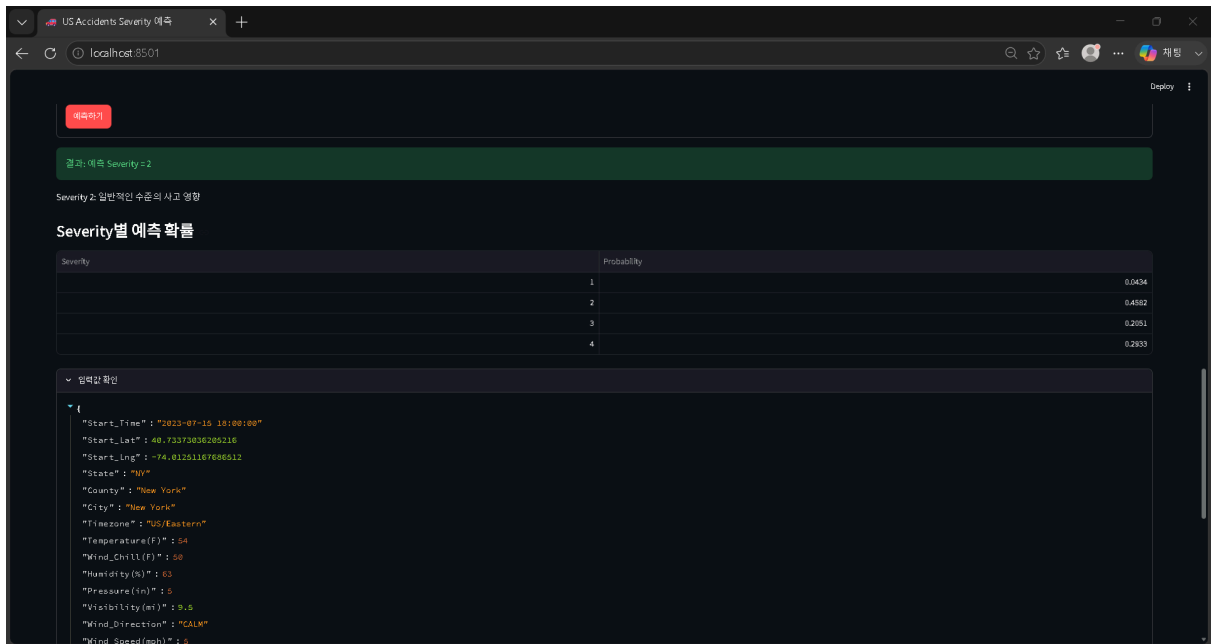


그림 6. Streamlit 웹앱의 **Severity** 예측 결과 화면

6. Analysis / Discussion

6.1 결과 해석

최종 모델인 RF Balanced min_samples_leaf=5는 Test set에서 Accuracy 0.7415, Weighted F1-score 0.7546을 기록하였다. 이는 전체 데이터 기준으로 비교적 안정적인 예측 성능을 보였음을 의미한다. 그러나 Balanced Accuracy는 0.4417, Macro F1-score는 0.4185로 Accuracy에 비해 낮게 나타났다. 이러한 차이는 데이터가 **Severity 2** 클래스에 크게 편중되어 있기 때문이다.

Classification Report를 보면 **Severity 2**의 F1-score는 0.8403으로 가장 높게 나타났다. 이는 전체 데이터에서 **Severity 2**가 차지하는 비중이 크기 때문에 모델이 해당 클래스의 패턴을 상대적으로 충분히 학습할 수 있었기 때문이다. 반면 **Severity 1**과 **Severity 4**의 F1-score는 각각 0.2006, 0.1630으로 낮게 나타났다. 두 클래스는 support가 작아 학습 데이터에서 충분한 사례를 확보하기 어려웠고, 그 결과 모델이 해당 클래스의 특징을 안정적으로 학습하지 못한 것으로 해석된다.

Confusion Matrix에서도 이러한 경향이 확인된다. 모델은 다수 클래스인 **Severity 2**를 중심으로 예측하는 경향을 보였으며, 소수 클래스인 **Severity 1**과 **Severity 4**는 다른 클래스로 오분류되는 경우가 많았다. 특히 사고 심각도는 **1, 2, 3, 4**로 구분되어 있지만 실제 데이터에서는 인접한 클래스 간 경계가 명확하지 않을 수 있다. 예를 들어 **Severity 2**와 **Severity 3**은 사고 영향 정도가 연속적인 특성을 가질 수 있어, 입력 변수만으로 명확히 구분하기 어려운 경우가 발생할 수 있다.

변수 중요도 분석에서는 **Start_Lng**, **Start_Lat**와 같은 위치 변수가 높은 중요도를 보였다. 이는 사고 **Severity**가 지역적 특성과 관련되어 있을 가능성을 보여준다. 또한 **Wind_Chill(F)**, **Temperature(F)**, **Pressure(in)**, **Humidity(%)**, **Hour**, **Month** 등 기상 및 시간 관련 변수도 상위권에 포함되었다. 이를 통해 사고 발생 위치, 시간대, 기상 조건이 사고 **Severity** 예측에 중요한 정보를 제공한 것으로 해석할 수 있다.

6.2 성능을 높이기 위해 시도한 점

Severity 예측 성능을 높이기 위해 여러 측면에서 접근을 시도하였다. 정보 누수 방지를 위한 변수 선택을 수행하였다. **End_Time**, **Distance(mi)**, **End_Lat**, **End_Lng**, **Description**과 같이 사고 종료 이후에 알 수 있는 변수를 제외함으로써, 모델이 비현실적인 정보에 의존하지 않고 사고 발생 시점의 조건만으로 학습하도록 하였다. 이는 검증 성능이 실제 예측 상황과 괴리되는 문제를 줄이는 데 기여하였다.

또한 다양한 파생 변수 생성과 전처리를 통해 입력 정보의 품질을 높였다. **Start_Time**에서 **Month**, **DayOfWeek**, **Hour**를 추출하여 사고 발생 시점의 시간적 특성을 반영하였고, 다양한 문자열로 존재하던 **Weather_Condition**을 **Weather_Group**으로 그룹화하여 희소 범주로 인한 노이즈를 완화하였다. 또한 **City**와 **County**처럼 고유값이 많은 범주형 변수는 상위 30개만 유지하고 나머지를 **Other**로 처리하여 입력 차원이 과도하게 증가하는 문제를 방지하였다.

다양한 모델과 클래스 불균형 처리 전략을 폭넓게 비교하여 최적의 조합을 탐색하였다. **Dummy Classifier**부터 **Logistic Regression**, **Decision Tree**, **Random Forest**, **HistGradientBoosting**, **XGBoost**, **LightGBM**, **Dense Neural Network**까지 여러 모델을 비교하였으며, **class_weight=balanced**, **manual class weight**, **undersampling**, **oversampling**, **SMOTE** 등 불균형 처리 방법을 함께 실험하였다. 이를 통해 단순히 하나의 모델에 의존하지

않고, **Validation Macro F1-score**를 기준으로 가장 안정적인 성능을 보이는 모델을 선정할 수 있었다.

마지막으로 트리 기반 앙상블 모델과 하이퍼파라미터 조정을 통해 일반화 성능을 확보하였다. 단일 **Decision Tree**의 과적합 문제를 완화하기 위해 여러 트리를 결합한 **Random Forest**를 활용하였고, **min_samples_leaf=5**를 적용하여 잎 노드가 지나치게 세분화되는 것을 막아 모델의 일반화 능력을 높였다. 또한 **class_weight=balanced**를 적용하여 소수 클래스의 오분류에 더 큰 패널티를 부여함으로써, 다수 클래스에 편향되는 경향을 일부 완화하였다.

6.3 성능이 낮게 나온 이유 및 실패 사례

반면 **Macro F1-score**와 **Balanced Accuracy**가 상대적으로 낮게 나타난 가장 큰 이유는 클래스 불균형이다. 본 데이터는 **Severity 2**가 대부분을 차지하고, **Severity 1**과 **Severity 4**는 데이터 수가 매우 적다. 모델은 학습 과정에서 자주 등장하는 클래스의 패턴을 더 많이 학습하게 되므로, 소수 클래스에 대한 예측 성능은 낮아질 수밖에 없다. 이는 **Confusion Matrix**에서 **Severity 1**과 **Severity 4**가 다른 클래스로 오분류되는 형태로 나타났다.

또 다른 원인은 입력 변수의 한계이다. 사고 **Severity**는 교통량, 평균 속도, 제한속도, 차종, 사고 유형, 충돌 형태, 차로 수, 도로 등급, 긴급차량 도착 시간 등과 밀접한 관련이 있을 수 있다. 그러나 **US Accidents** 데이터에는 이러한 교통공학적 핵심 변수가 충분히 포함되어 있지 않았다. 따라서 모델은 주어진 시간, 위치, 기상, 도로환경 변수만으로 **Severity**를 예측해야 했고, 이로 인해 사고 심각도를 정밀하게 구분하는 데 한계가 발생하였다.

또한 **Severity** 자체가 사고의 물리적 피해 정도라기보다 사고로 인한 교통 영향 또는 지연 정도를 나타내는 변수라는 점도 한계로 작용할 수 있다. 예를 들어 비슷한 사고 조건이라도 도시부와 지방부, 혼잡 시간대와 비혼잡 시간대, 주요 간선도로와 일반도로에 따라 교통 영향은 다르게 나타날 수 있다. 하지만 현재 입력 변수만으로 이러한 맥락을 충분히 반영하기는 어렵다.

실패 사례는 주로 소수 클래스에서 발생하였다. **Severity 1** 또는 **Severity 4**에 해당하는 사고가 **Severity 2**나 **Severity 3**으로 예측되는 경우가 많았다. 이는 모델이 극단적으로

낮거나 높은 **Severity**를 구분할 만큼 충분한 학습 사례를 확보하지 못했기 때문이다. 또한 인접 등급 간 구분이 애매한 사고에서는 모델이 중간 등급으로 예측하는 경향이 나타날 수 있다.

6.4 한계점

데이터 불균형

Severity 2 중심의 데이터 구조는 모델이 다수 클래스에 편향되도록 만들며, 소수 클래스의 예측 성능을 제한한다. **Class weight**, **oversampling**, **SMOTE** 등의 방법을 적용했음에도 불구하고 소수 클래스 성능 개선에는 한계가 있었다

입력 변수의 제한

사고 심각도에 직접적인 영향을 줄 수 있는 교통량, 속도, 차종, 사고 유형, 도로 등급, 제한속도 등의 변수가 포함되어 있지 않아 모델의 설명력과 예측 성능이 제한되었다. 특히 사고의 실제 피해 정도나 현장 상황을 나타내는 변수가 부족하기 때문에, 모델이 **Severity**를 간접적인 환경 변수로만 추정해야 했다.

데이터 분할 방식

Severity 비율을 유지하기 위해 **stratified random split**을 사용하였다. 이 방식은 클래스 비율을 안정적으로 유지할 수 있다는 장점이 있지만, 유사한 지역이나 시간대의 사고가 **Train**, **Validation**, **Test set**에 함께 포함될 수 있다. 따라서 현재 성능은 동일한 데이터 분포 내에서의 일반화 성능으로 해석하는 것이 적절하며, 새로운 지역이나 미래 시점에 대한 예측 성능을 보장한다고 보기는 어렵다.

웹앱 입력 방식

Streamlit 웹앱에서는 사용자가 직접 기상 및 도로환경 정보를 입력해야 하며, 지도 클릭 시 행정구역 정보는 가장 가까운 기존 사고 데이터를 기준으로 자동 설정된다. 이는 외부 **API**를 사용하지 않아 실행이 간단하다는 장점이 있지만, 실제 행정구역과 완전히 일치하지 않을 수 있다.

6.5 개선 방향

외부 교통·도로 데이터 결합의 필요성이다. 교통량, 평균 속도, 제한속도, 도로 등급, 차로 수, 사고 유형, 차량 종류 등의 변수를 추가하면 사고 **Severity**를 더 직접적으로 설명할 수 있을 것이다. 특히 교통량과 속도 정보는 사고로 인한 지연 정도와 밀접하게 관련될 수 있으므로 모델 성능 향상에 도움이 될 가능성이 높다.

클래스 불균형 문제를 더 고도화하여 처리할 필요가 있다. 본 프로젝트에서는 **class weight**, **undersampling**, **oversampling**, **SMOTE** 등을 실험하였지만, 소수 클래스 성능 개선에는 한계가 있었다. 향후에는 **BalancedRandomForest**, **EasyEnsemble**, **SMOTENC**와 같이 불균형 정형 데이터에 적합한 방법을 추가로 적용할 수 있다. 특히 범주형 변수가 많은 데이터에서는 단순 **SMOTE**보다 범주형 변수를 고려하는 **SMOTENC**가 더 적절할 수 있다.

문제 정의를 재구성하는 방법도 고려할 수 있다. 현재는 **Severity 1, 2, 3, 4**의 4분류 문제로 접근했지만, 소수 클래스의 데이터 수가 매우 적어 예측이 어렵다. 따라서 **Low / High**의 2분류 또는 **Low / Medium / High**의 3분류로 재정의하면 클래스 간 경계를 더 명확하게 만들고 실용적인 예측 성능을 높일 수 있다.

검증 방식을 개선할 필요가 있다. 향후에는 **State** 단위 분리 검증, 도시 단위 분리 검증, 시간 기준 분리 검증을 적용하여 새로운 지역이나 미래 시점에 대한 일반화 성능을 확인할 수 있다. 이를 통해 모델이 단순히 기존 데이터 분포를 학습한 것인지, 실제 새로운 환경에서도 적용 가능한지를 더 엄밀하게 평가할 수 있다.

웹앱 기능을 확장할 수 있다. 현재 웹앱은 사용자가 직접 입력한 사고 조건을 기반으로 **Severity**를 예측한다. 향후에는 날씨 **API**, 지도 **API**, 교통량 **API**와 연계하여 사용자가 위치와 시간만 입력해도 기상 및 도로 정보를 자동으로 불러오는 방식으로 개선할 수 있다. 이를 통해 입력 편의성을 높이고 실제 활용 가능성을 강화할 수 있다.

한계점	영향	개선 방향
Severity 2 중심 클래스 불균형	소수 클래스 예측 성능 저하	BalancedRandomForest , EasyEnsemble , SMOTENC 적용
교통량·속도·차종 변수 부재	사고 심각도 설명력 제한	외부 교통·도로 데이터 결합
Stratified random split 사용	지역·시간 일반화 성능 검증 부족	State 단위 또는 시간 기준 분리 검증

4분류 클래스 경계 모호	인접 클래스 간 오분류 발생	2분류 또는 3분류 문제 재정의
웹앱 수동 입력 부담	실제 사용 편의성 제한	날씨·지도·교통 API 연계

표 8. 한계점 및 개선 방향 요약

7. Conclusion

7.1 전체 프로젝트 요약

Kaggle의 US Accidents (2016–2023) 데이터를 활용하여 교통사고 발생 조건 기반의 사고 심각도(Severity) 예측 모델을 개발하는 것을 목표로 하였다. 사고 발생 시간, 위치, 기상 조건, 주야간 정보, 도로환경 변수를 입력 변수로 사용하였으며, Severity 1, 2, 3, 4를 예측하는 다중분류 문제로 정의하였다.

실제 사고 발생 시점에서 활용 가능한 정보만을 모델 입력으로 사용하기 위해 End_Time, Distance(mi), End_Lat, End_Lng, Description과 같이 사고 종료 이후에 알 수 있거나 사고 결과와 직접적으로 연결될 수 있는 변수는 제외하였다. 이후 시간 파생 변수 생성, 날씨 그룹화, 결측치 처리, 범주형 변수 인코딩, 수치형 변수 표준화 등의 전처리를 수행하였다.

모델링 단계에서는 Dummy Classifier, Logistic Regression, Decision Tree, Random Forest, HistGradientBoosting, XGBoost, LightGBM, Dense Neural Network 등 다양한 모델을 비교하였다. 또한 Severity 2 중심의 클래스 불균형 문제를 고려하여 class weight, undersampling, oversampling, SMOTE 등의 불균형 처리 방법을 실험하였다. 최종 모델은 Validation set의 Macro F1-score를 기준으로 선정하였으며, RF Balanced min_samples_leaf=5 모델이 가장 우수한 성능을 보여 최종 모델로 선택되었다.

최종 모델은 Test set에서 Accuracy 0.7415, Balanced Accuracy 0.4417, Macro F1-score 0.4185, Weighted F1-score 0.7546을 기록하였다. 전체 정답률과 Weighted F1-score는 비교적 안정적으로 나타났지만, Macro F1-score와 Balanced Accuracy는 상대적으로 낮았다. 이는 데이터가 Severity 2에 크게 편중되어 있고, Severity 1과 Severity 4의 support가 작아 소수 클래스 예측이 어려웠기 때문이다.

또한 학습된 최종 모델과 전처리 정보를 저장하여 Streamlit 기반 웹앱으로 연결하였다. 웹앱에서는 사용자가 미국 지도에서 사고 위치를 선택하고, 기상 정보와 도로환경 정보를

입력하면 예측 **Severity**와 클래스별 확률을 확인할 수 있도록 구현하였다. 이를 통해 단순한 모델 학습을 넘어, 사용자가 직접 예측 결과를 확인할 수 있는 형태의 응용 시스템까지 구현하였다.

7.2 기대 효과 및 향후 확장 가능성

기대 효과는 교통사고 발생 조건을 활용하여 사고 **Severity**를 사전에 예측할 수 있는 가능성을 확인했다는 점이다. 사고 발생 직후 **Severity**를 예측할 수 있다면 사고 대응 우선순위 설정, 교통관리 전략 수립, 위험 지역 모니터링 등에 활용될 수 있다. 특히 위치, 시간, 기상, 도로환경 정보를 함께 고려한 예측 모델은 교통 운영자가 사고의 잠재적 영향을 판단하는 보조 도구로 활용될 수 있다.

또한 **Streamlit** 웹앱 구현을 통해 모델 결과를 실제 사용 가능한 형태로 확장하였다. 사용자는 지도 기반으로 사고 위치를 선택하고, 주요 사고 조건을 입력하여 예측 결과를 확인할 수 있다. 이러한 구조는 향후 공공 교통 데이터, 기상 **API**, 지도 **API**, 교통량 **API**와 연계될 경우 실제 교통관리 지원 시스템으로 확장될 수 있다.

향후에는 교통량, 평균 속도, 제한속도, 도로 등급, 차로 수, 차종, 사고 유형과 같은 외부 교통·도로 데이터를 결합할 필요가 있다. 이러한 변수는 사고 **Severity**를 더 직접적으로 설명할 수 있으므로 소수 클래스 예측 성능 개선에 도움이 될 수 있다. 또한 현재의 4분류 문제를 **Low / High** 형태의 2분류 또는 **Low / Medium / High** 형태의 3분류로 재정의하면 클래스 간 경계를 더 명확히 하고 실용적인 예측 성능을 높일 수 있다.

검증 방식 역시 개선할 필요가 있다. 본 프로젝트에서는 **Severity** 비율을 유지하기 위해 **stratified random split**을 사용하였지만, 이 방식은 유사한 지역이나 시간대의 사고가 **Train**, **Validation**, **Test set**에 함께 포함될 수 있다. 향후에는 **State** 단위 분리 검증, 도시 단위 분리 검증, 시간 기준 분리 검증을 적용하여 새로운 지역이나 미래 시점에 대한 일반화 성능을 더 엄밀하게 평가할 필요가 있다.

종합적으로 본 프로젝트는 **US Accidents** 데이터를 활용하여 교통사고 **Severity** 예측 모델을 구축하고, 이를 웹앱으로 구현한 실습형 머신러닝 프로젝트이다. 데이터 불균형과 입력 변수 부족으로 인한 한계는 존재하지만, 정보 누수를 고려한 변수 선택, **Macro F1-score** 중심의 모델 평가, 최종 모델의 웹앱 구현까지 하나의 파이프라인으로 연결했다는 점에서 의의가 있다.

AI 활용 내역

본 보고서 작성 과정에서 문장 정리 및 맞춤법 검사, 보고서 구조 점검, 그림 1의 파이프라인 시각화 보조에 생성형 AI를 활용하였다. 모델 학습, 실험 결과 산출, 성능 평가, 웹앱 구현은 팀이 작성한 코드와 실험 결과를 기반으로 수행하였다.

References

- [1] S. Moosavi, "US Accidents (2016–2023)," Kaggle Dataset. Available: Kaggle, US Accidents (2016–2023).
- [2] M. A. Sufian and J. Varadarajan, "Enhancing Prediction and Analysis of UK Road Traffic Accident Severity Using AI: Integration of Machine Learning, Econometric Techniques, and Time Series Forecasting in Public Health Research," arXiv preprint arXiv:2309.13483, 2023.
- [3] A. Adefabi, S. Olisah, C. Obunadike, O. Oyetubo, E. Taiwo, and E. Tella, "Predicting Accident Severity: An Analysis of Factors Affecting Accident Severity Using Random Forest Model," arXiv preprint arXiv:2310.05840, 2023.
- [4] S. Somvanshi, P. Hebli, G. Chhetri, and S. Das, "Tabular Data with Class Imbalance: Predicting Electric Vehicle Crash Severity with Pretrained Transformers (TabPFN) and Mamba-Based Models," arXiv preprint arXiv:2509.11449, 2025.